Chair of IT Security
Brandenburg University of Technology

Asya Mitseva, M.Sc.
Prof. Dr.-Ing. Andriy Panchenko

Winter Semester 2025/2026

Part IV: Comparison of Detection Methods **Deadline: 08.01.2025**

Study Project: Adversarial Machine Learning
Use of Reverse Engineering for Anomaly Detection in Industrial Control Systems

Introduction

Industrial Control Systems (ICS) monitor and control critical infrastructures such as chemical plants, power systems, gas pipelines, and water systems. Typically, ICS consist of two macro areas, known as Information Technology (IT) and Operational Technology (OT). While IT is composed of standard computers interconnected via traditional communication protocols, OT includes special hardware and software for monitoring and controlling a given industrial facility. Overall, OT consists of *field*, *control*, and *supervisory* layers. The field layer includes different sensors and actuators, while the control layer is composed of a number of programmable logic controllers (PLCs). The sensors measure the current state of the underlying physical process, which is then reported to the PLCs. The PLCs implement a control logic used to evaluate the obtained sensor readings according to predefined or dynamically adjustable target values. Based on this evaluation, the PLCs may initiate specific commands to one or more actuators in the field layer aiming to adjust the current state of the physical process. In the supervisory layer, a Supervisory Control and Data Acquisition (SCADA) system performs a high-level visualization and control over the control and field layers and provides an overview of the current process state to the operator. While OT was mainly a standalone system in the past, nowadays it incorporates both decades-old devices and communication infrastructure, and new generation embedded devices with computing capabilities and Ethernet-based communication protocols, and is more often connected to the Internet. While the digitization of ICS makes the monitoring and the control of the physical process easier, it creates new attack surfaces against ICS.

A major line of research focused on the design and the development of robust intrusion detection techniques for the identification of cyber attacks in ICS. A particular challenge to be addressed is the usage of non-standardized protocols that are specific to manufactures or even to a particular type of device. Consequently, protocol specifications needed to process network data are not available and, thus, an in-depth analysis of the content of exchanged network traffic is difficult to accomplish.

The goal of this study project is to implement and evaluate different approaches that are able to extract expressive patterns, i.e., *features*, from network traffic collected in the OT area of a given ICS. We assume that the ICS operator utilizes proprietary binary protocols, i.e., the approaches to be developed cannot directly parse the content of transmitted network packets, and the defender relies on reverse engineering methods to derive only specific chunks from the observed network packets. The defender uses different machine learning (ML) methods with these chunks to identify a possible abnormal operation in ICS.

In the previous practical sheet, we implemented another detection method and prepared different machine learning algorithms for anomaly detection. In the next part of the study project, we continue with the evaluation of the performance of these methods using the features generated from your autoencoder.

Task 1: Feature Importance, Analysis of Prediction Errors, Complexity Analysis

The goal of this task is to establish the importance of the different features generated through the autoencoder and to take a closer look at the classification errors produced by each of the classifiers from the experiments executed in Task 3 of the previous practical sheet. In this task, you should write a piece of code that executes the following analysis:

a) For each of the implemented ML classifiers and for each of the evaluated scenarios, you should establish the importance score of each of your feature sets and plot the obtained results, always starting with the most important features.

b) For the set of experiments in each of the items in Task 3 of the previous practical sheet, you should implement and plot a Venn diagram of classification errors for each of the different types of features and for each of the evaluation scenarios. Each circle should represent the set of prediction errors for one of your ML-based classifiers. The circles should intersect when the same testing instance is incorrectly classified by the different classifiers.

c) For each of the implemented classifiers and each of the experiments from Task 3 of the previous practical sheet, you should measure the the runtime and the memory needed during the feature generation, the training phase, and the testing phase, respectively.

Plot all obtained evaluation results in an appropriate way and submit your plots in Moodle. Describe in details all takeaways that can be concluded after executing the experiments.

Task 2: GAN-based Anomaly Detector

ME

The goal of this task is to write a piece of code that implements another deep learning (DL) based detection method classifying each input from a given dataset as either being benign, i.e. normal, or malicious, i.e., containing an attack. In particular, you should implement a Generative Adversarial Network (GAN) that takes m x n values as an input and returns a classification result / decision as an output. The GAN should be able to work with network packets as an input data, i.e., it should take as an input m x n network packets, where m indicates the number of packets and n the set of bytes of each packet. In particular, you need to write a piece of code that satisfies the following requirements:

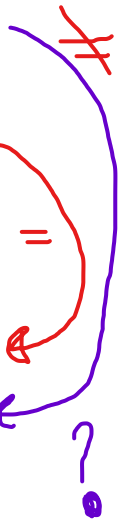
a) Your GAN model should take as an input a predefined set of bytes n_p per network packet such that $n_p = M$ values and M should be a parameter that is read from the command line.

b) Your GAN should consist of two components: a discriminator and a generator. The discriminator should be a two-class classifier that is trained to differentiate between real data samples and fake data samples that are generated by the generator. The generator should be trained to create those fake data samples in a way that is as hard to distinguish from the real samples as possible.

c) Your discriminator should be a convolutional neural network (CNN) that consists of 9 convolutional layers with a ReLU activation function. In addition, you should add a dropout layer after all convolutional layers except the first and last layers. Then, the output from the last convolutional layer should be passed to 4 fully connected layers with a ReLU activation function. The discriminator should output a single value in the range $[0, 1)$ indicating how real the input data is.

d) Your generator should be given a noise data as input and should operate as a reverse CNN that uses a method, called deconvolution, to transform the noise into meaningful network packet bytes. In particular, the generator should be initiated with 4 fully connected layers with a LeakyReLU activation function that are followed by 8 convolutional layers with a LeakyReLU activation function. In addition, the first 4 layers from these 8 convolutional layers should be preceded by an upsampling layer. The function used to generate the input noise data for the generator is of your choice.

Keep it as is, if I can not create for any reason, make it 9, but we should justify why.



e) For your generator, you should implement custom loss function that is computed on the input used for the first fully connected layer of the discriminator. This loss function should seek minimizing the difference between the features found in real data and the features found in its generated samples.

f) Your GAN should support the following two anomaly detection options: (i) after training, the distriminator is used to detect possible anomalies during testing, or (ii) after training, the generator is used to detect possible anomalies during testing. The user should be able to choose from the command line which of both options to use.

g) Finally, you should write piece of code that can perform hyperparameter tuning for your GAN, e.g., optimization of the parameters needed for your convolutional layers, number of training epochs, etc.

Task 3: Execution of Experiments (Part 2)

The goal of this task is to continue with the evaluation of your detection methods by using the features created through the autoencoder for each of the evaluation scenarios using cross-validation, defined in the previous practical sheet, and by using the corresponding training and testing sets. In particular, you should execute the following experiments:

a) For the first evaluation scenario, you should execute four experiments with your N-gram based detection method implemented in the previous practical sheet with $N = 2$, in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the parameters and compute the *precision* and *recall* for each of the experiments.

b) For the first evaluation scenario, you should execute four experiments with your N-gram based detection method implemented in the previous practical sheet with $N = 5$, in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the parameters and compute the *precision* and *recall* for each of the experiments.

c) For the first evaluation scenario, you should execute four experiments with your autoencoder used as a classifier that was implemented in the first practical sheet, in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the parameters and compute the *precision* and *recall* for each of the experiments.

d) For the first evaluation scenario, you should execute four experiments with your GAN method using the discriminator for anomaly detection, in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the parameters and compute the *precision* and *recall* for each of the experiments.

e) For the first evaluation scenario, you should execute four experiments with your GAN method using the generator for anomaly detection, in which you use either (i) entire network packets without applying the reverse engineering as input or (ii) parts of network packets for a sequence p of physical readings where $p = \{5, 10, 15\}$ after applying reverse engineering. For this item, you should use the raw bytes of the packets, not the features generated by the autoencoder. Make sure that for the detection method you identify and use the parameters and compute the *precision* and *recall* for each of the experiments.

Plot all obtained classification results in an appropriate way and submit your plots and the obtained classification results in Moodle. Describe in details all takeaways that can be concluded after executing the experiments.

Preparation for Q&A Session

Prepare yourself for a Q&A session of 40 minutes. During the Q&A session, you need to give a short overview of libraries, scripts or already existing tools that you have used. Furthermore, you need to make a short demo of your piece of code, explain its operation in detail, and show all classification results and plots created. Last but not least, you should be ready to answer spontaneous questions asked by the reviewer.